This article introduces application development using the Qt GUI framework.

Part—1

There was a time when all desktop applications were developed from scratch. Then came the concept of code re-use. Static and shared libraries were created for use in application development, but developers didn't stop at that; they came up with software frameworks. Though libraries and frameworks seem to functionally be the same, there are some major differences between them, which should be understood:

- Libraries offer re-usability of functionality, whereas frameworks offer re-usability of behaviour. For example, a library may provide classes for TCP and UDP sockets, while a framework will provide a class for an abstract socket.
- A library may provide functions used for signals/events, but the framework function is how they interact with the system and other components.
- Libraries are called from application code, but the framework calls application code—or, you can say, provides services to the code.

Desktop applications

Desktop applications are very platform-specific. An application compiled for Linux cannot be directly executed on another OS without some sort of emulation, due to differences in system calls and libraries between OSs. A big question was how to write platform-independent applications. One method was to develop libraries for each platform, keeping application code the same and re-compiling for each target platform. This does make life easier for developers—but then everything changed with the revolutionary concept of virtual machines from Sun Microsystems, which gave the world the platform-independent Java programming language. Java applications run on the Java Virtual Machine (JVM). Code developed once can be deployed to every platform that has a JVM for it.

But everything comes with a price. In Java's case, the price was application performance. The performance of Java applications is not as good as of C/C++ applications compiled to platform-native code. Another problem is the large memory footprint. No doubt Java is still a leading language, but these days we have some really big data applications (for example, in bio-tech) with very high performance requirements. So again, the need is to develop applications compilable to native code. Application development using C is time-consuming, while C++ is a better option. We have some frameworks that support C++ for application development. The Qt framework is one of them.

Qt

Qt is a cross-platform application framework developed by Trolltech and presently owned by Nokia *(qt.nokia.com)*. Its APIs are for C++. Qt has been extensively used by application developers to develop cross-platform applications.

Qt can help with graphical application development, network applications, database and multimedia applications, handling XML and 3D, painting, drawing and Web access. As far as platform support is concerned, it supports Linux, Mac OS, Windows, Meego, Embedded Linux and Symbian.

Qt architecture

In Figure 1, you can see a minimal architecture diagram. The top layer is C++ program code. Below that are Qt classes for GUI, WebKit, databases, etc, and then an OS-specific support layer. Earlier, Qt also supported Java; its Java-based version was called Jambie. As Qt development progressed, it was getting difficult to support both C++ and Java, so the decision was made to support only C++.

Qt installation

Installation methods differ by OS. On Ubuntu 10.04 LTS, I installed Qt via the Synaptic package manager—install the *'qtcreator'* package; dependency resolution will install Qt Assistant, Qt Designer, Qt Linguist and Qt Creator.

If you want to try out the latest Qt release, you can download the offline installer from *http://qt.nokia.com/downloads* and install it. Just *chmod* the installer file to make it executable and run it, and then follow the prompts.

A 'Hello World' program (non-GUI)

Open a terminal. Create a directory *(first)* and create a simple 'Hello World' program with the following code:

```
#include<QtCore>
int main(){
      qDebug() << "Hello world\n";
}
```

The included header file *QtCore* contains declarations for classes that do not use a GUI. We will look at these in detail in later articles. For now, just use the *qDebug* class, which outputs debugging messages to the console and is equivalent